

Live Virtual Machine Migration based on Improved Pre-copy Approach

Fei Ma, Feng Liu, Zhen Liu ^{*†}

^{*} Network Management Research Centre, Beijing Jiaotong University

[†] Engineering Research Center of Network Management Technology for High Speed Railway, Ministry of Education
Beijing 100044, China
mafei2007@gmail.com

Abstract—Live migration of virtual machine has attracted significant attention in recent years. It facilitates system online maintenance, load balancing, fault tolerance and power management. Although pre-copy approach can balance the downtime and total migration time of contradictions, it will cap the number of copying iterations to the maximum number of iterations since the writable working set is not guaranteed to converge across successive iterations, especially when the VM is executing a largely read-intensive workload. In this paper, we improve the pre-copy approach on Xen 3.3.0. We add a bitmap page in our improved pre-copy approach, which marks those frequently updated pages. By the judge in the iteration process, we put those frequently updated pages into the page bitmap, and those pages can only be transmitted in the last round of the iteration process. This can ensure that those frequently updated pages are transmitted just once in the iteration process. Experiment demonstrates that compared with pre-copy, our improved pre-copy approach can reduce 34% of total transferred data and 32.5% of total migration time total transferred data on average. More importantly, our approach can complete the iteration process no more than 5 times.

Keywords—virtual machine; live migration; improved pre-copy approach;

I. INTRODUCTION

Virtual machine (VM) [1] is usually used in data centers, cluster computing and so on. It allows multiple operating systems run on the same physical machine, provides reliable isolation, and greatly increases the utilization of physical resources.

Live migration is a key selling point for state-of-the-art virtualization technologies. It has attracted significant attention in recent years [2] [3] [9] [10] [11]. Live migration of virtual machine relocates the memory and virtual device state of a VM from one physical machine to another, with no noticeable downtime of the VM. This offers interesting advantages for data center management, including [13]:

- 1) **Load balancing:** VMs can be dynamically migrated depending on workload, offering more efficient usage of computing resources.
- 2) **Power efficiency:** VMs with low workload can be consolidated to fewer physical machines, allowing to power off resources.

- 3) **Transparent infrastructure maintenance:** Before machines are brought down for maintenance, administrators can relocate VMs to other nodes without any noticeable interruption of service for users.

In this paper, we focus on VM migration within a cluster environment where a network-accessible storage system (such as SAN or NAS) is employed. Only memory and CPU status needs to be transferred from the source node to the target one.

Live migration techniques in the state of the art mainly use pre-copy approach [2] [3], which first transfers all memory pages and then copies pages just modified during the last round iteratively. When applications' writable working set (WWS) becomes small or the maximum number of iterations is reached, the virtual machine is suspended and only CPU state and dirty pages in the last round are sent out to the destination.

Although pre-copy approach can balance the downtime and total migration time of contradictions, it will cap the number of copying iterations to the maximum number of iterations since the WWS is not guaranteed to converge across successive iterations, especially when the VM is executing a largely read-intensive workload. Moreover, even moderately write-intensive workloads can reduce pre-copy's effectiveness during migration [9].

In this paper, we improve the pre-copy approach on Xen 3.3.0. We add a bitmap page in our improved pre-copy approach, which marks those frequently updated pages. By the judge in the iteration process, we put those frequently updated pages into this bitmap page, and those pages can only be transmitted in the last round of the iteration process. This can ensure that those frequently updated pages are transmitted just once in the iteration process, thus avoiding the duplicate transmission overhead of pre-copy.

This paper is organized as follows. In section 2, we cover the relate work. Then, in section 3, we describe the design of our improved pre-copy approach, and present the experimental results in section 4. Finally, we conclude and give our future work in section 5.

II. RELATED WORKS

Pre-copy is the prevailing live migration technique to perform live migration of VMs. These include hypervisor-

based approaches such as VMware [3], Xen [4], and KVM [5], operating-system level approaches such as OpenVZ system [6], as well as migration over wide-area network [7] [8]. All of the above systems use pre-copy algorithm for live VM migration in a memory-to-memory approach.

Hines et al. [9] propose using post-copy instead of pre-copy for live migration. Post-copy migration defers the memory transfer phase until after the VM's CPU state has already been transferred to the target and resumed there. Post-copy first transmits all processor state to the target, starts the VM at the target, and then actively pushes memory pages from source to target. Concurrently, any memory pages that are faulted on by the VM at target, and not yet pushed, are demand-paged over the network from source. But the downtime is much higher than that of pre-copy due to the latency of fetching pages from the source node before VM can be resumed on the target.

Liu et al. [10] advocate the use of checkpointing/recovery and trace/replay instead of pre-copy. Checkpointing/recovery and trace/replay technology are adopted to provide fast, transparent VM migration in a LAN. A trace daemon continuously logs the non-deterministic events of the VM while sacrificing very little performance. The execution trace file logged at the source host is iteratively transferred to the target host and used to synchronize the migrated VM's execution state. They show that it allows reduce downtime, total migration time and total data transferred. But it is valid only when the log replay rate is larger than the log growth rate.

Jin et al. [11] propose using adaptive compression of migrated data: different compression algorithms are chosen depending on characteristics of memory pages. They first use memory compression to provide fast VM migration, and they also design a zero-aware characteristics-based compression (CBC) algorithm for live migration. In the source node, data being transferred in each round are first compressed by their algorithm. When arriving on the target, compressed data are then decompressed. However, memory compression increases the system overhead.

III. DESIGN OF IMPROVED PRE-COPY APPROACH

In this section, we first introduce the three metrics in live virtual machine migration: downtime, total migration time and total data transmitted. Then, we describe the detailed improved pre-copy approach and the migration procedure, followed by the analysis between the pre-copy approach and our improved pre-copy approach.

Live virtual machine migration takes a running VM and moves it from one physical machine to another. The performance of any live virtual machine migration strategy could be gauged by the following metrics.

- 1) **Downtime:** the time when no CPU cycle is devoted to any of the VM-resident applications, neither at the source nor at the target system, it consists of the time necessary to suspend the VM on the source, transfer the VM state to the destination, load the device state, and activate the migrated VM on the remote host.

- 2) **Total Migration Time:** the duration between the time the migration is initiated and the time the migrated VM gets a consistent state with the original one, i.e., during which the state of two VMs is synchronized.
- 3) **Total Data Transmitted:** the total data is transferred while synchronizing the both VMs' state [10].

A. Improved pre-copy approach

According to the used state of the page, the memory page in Xen which will be migrated is divided into three categories, i.e. three kinds of bitmap page of Xen: TO_SEND, TO_SKIP and TO_FIX. The three kinds of bitmap page are described as following:

- **TO_SEND:** marked the pages which get dirty in the previous iteration process, i.e. the pages which need to be transmitted in this iteration.
- **TO_SKIP:** marked the pages which can be skipped in this iteration.
- **TO_FIX:** marked the pages which need to be transmitted in the last iteration.

In fact, the pages in the TO_SEND bitmap page are the pages which are modified in the previous iteration process. Those frequently updated pages recorded in TO_SKIP bitmap page, and those pages which have not been mapped, in TO_FIX bitmap page, and be transmitted in the last round of the iteration process.

And, we add a bitmap page named TO_SEND_LAST in our improved pre-copy approach. As the TO_FIX bitmap page, TO_SEND_LAST marks the pages which need to be transmitted in the last iteration. However, unlike TO_FIX, TO_SEND_LAST records those frequently updated pages.

How to determine those frequently updated pages? In order to facilitate control, we think if it appears a retransmission of the page, the page should be put into the TO_SEND_LAST bitmap page, i.e. the page which appears in the TO_SKIP bitmap page should also be put into the TO_SEND_LAST bitmap page.

The improved pre-copy migration procedure as Xen, includes the following stages in turn: pre-migration, reservation, iterative pre-copy, stop and copy, commitment and activation [2]. The stages stop and copy and commitment contribute to the downtime of VM migration, while total migration time includes downtime and the duration of the stage iterative pre-copy. Our improved pre-copy approach is mainly in iterative pre-copy, which first transfers all memory pages, then puts those frequently updated pages into the TO_SEND_LAST bitmap page, and sends those pages out in the last round.

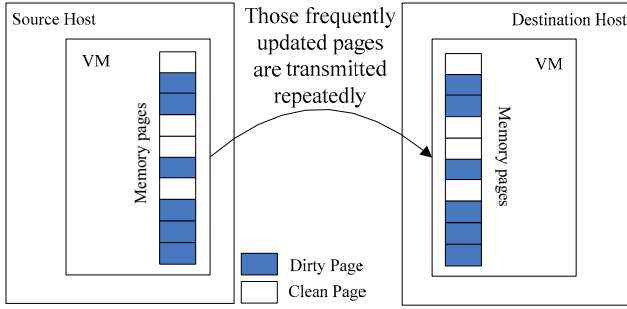
B. Analysis

As we know, in live migration the iterative copying of dirtied pages continues until either a small, writable working set has been identified, or a preset number of iterations is reached, whichever comes first.

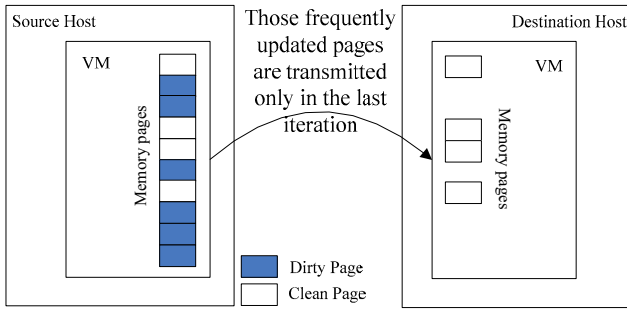
In pre-copy approach, those frequently updated pages are transmitted repeated, so the WWS is not guaranteed to

converge across successive iterations, and it will cap the number of copying iterations to the maximum number of iterations, especially when the VM is executing a largely read-intensive workload. As is shown in Figure 1 (a), those frequently updated pages are transmitted repeatedly in pre-copy approach of live migration.

However, in our improved pre-copy approach, those frequently updated pages are put into the TO_SEND_LAST bitmap page, and transmitted only in the last iteration, so the WWS can converge quickly, and the iterative process can be finished in several iterations. As is shown in Figure 1 (b), those frequently updated pages are transmitted only in the last iteration in improved pre-copy approach of live migration.



(a) One iteration in pre-copy live migration



(b) One iteration in improved pre-copy live migration

Figure 1. Two iterations in different approach of live migration

IV. EVALUATIONS

In this section, we present a detailed evaluation of our improved pre-copy implementation and compare it against Xen's pre-copy migration.

A. Experimental Setup

Our test environment consists of two 2.66 GHz dual core Intel machines connected via a Fast Ethernet switch. Each machine has 4 GB of memory. We use Ubuntu 8.04 as the guest OS and the privileged domain OS (domain 0). The host kernel is the modified version of Xen 3.3.0. Guest VM sizes range from 64 MB to 1024 MB. In order to increase the load on the virtual machine, we run a MUMmer [12] program in virtual machine. MUMmer is a scientific application to rapidly align genomes that has a very intensive memory usage.

The migrated virtual machine as the only guest one resides on the source node and there are no any guest virtual machine on the target machine. In the migration process, we evaluate

the same four primary metrics: iterations, total pages transferred, total migration time and downtime. Each experiment is repeated three times and every test result comes from the arithmetic average of three values.

B. Experimental Results

1) Iterations

Iterations is the number of iteration process. Iterations is a key performance metrics in live migration, because the fewer number of iterations, the less the total migration time.

Table I shows that our improved pre-copy approach dramatically reduces the time of iteration, compared with pre-copy approach. Pre-copy always reach 30, which is the maximum number of iterations, but improved pre-copy no more than 5 times.

TABLE I. ITERATIONS OF PRE-COPY AND IMPROVED PRE-COPY FOR DIVERSE VM MEMORY SIZE

VM Memory Size	64M	128M	256M	512M	1024M
Pre-copy Iterations	30	30	30	30	30
Improved pre-copy's Iterations	4	4	5	5	4

2) Total Data Transferred

Figure 2 shows that, compared with pre-copy, improved pre-copy has greater performance improvement on total data transferred. This should be attributed to smaller rounds and less data transferred in each round. Experimental results show that improved pre-copy reduces network traffic of VM migration by at least 10% (1024M) and at most 63% (64M), an average of 34%. Low-bandwidth wide area networks (WAN) would benefit greatly from our improved pre-copy approach.

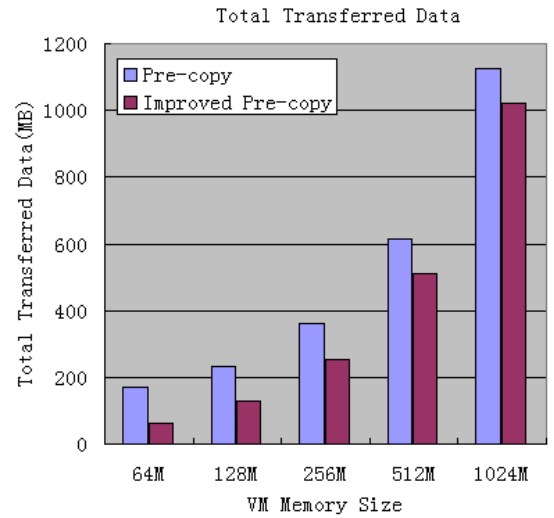


Figure 2. Total transferred data of Pre-copy and Improved Pre-copy for diverse VM memory size

3) Total Migration Time and Downtime

Figure 3 shows that improved pre-copy reduces the total migration time for diverse VM memory size, when compared to pre-copy, the same as total data transferred. Improved pre-

copy reduces total migration time by an average of 32.5%. In clusters or data centers, less total migration time of VMs would be more attractive to administrators.

However, the downtime in Figure 4 is currently much higher for improved pre-copy than for pre-copy. That is because those duplicate pages were placed in the last round of the transmission. Nevertheless, this tradeoff between total migration time and downtime may be acceptable in situations where network overhead needs to be kept low and the entire migration needs to be completed quickly.

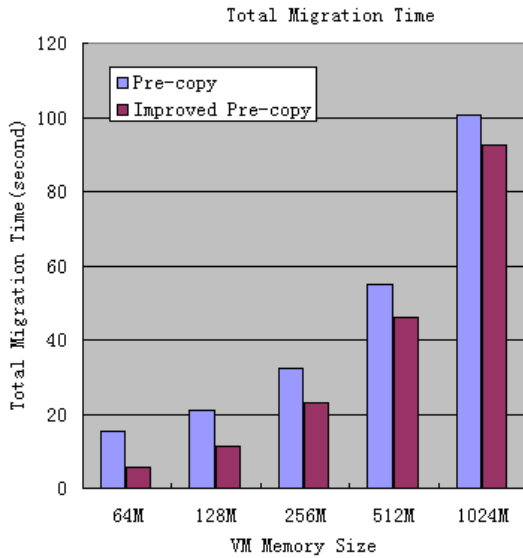


Figure 3. Total migration time of Pre-copy and Improved Pre-copy for diverse VM memory size

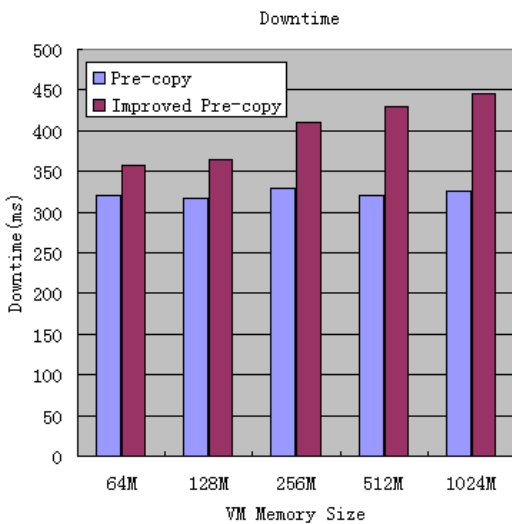


Figure 4. The downtime of Pre-copy and Improved Pre-copy for diverse VM memory size

V. CONCLUSIONS AND FUTURE WORK

In this paper, we have presented the design and evaluation of our improved pre-copy approach based on Xen 3.3.0. Those frequently updated pages are transmitted in the last round of the iteration process and only once. Experiment demonstrates that compared with pre-copy, our improved pre-copy approach can reduce 34% of total transferred data and 32.5% of total migration time total transferred data on average. More importantly, our approach can complete the iteration process no more than 5 times. In future work, we intend to use our improved pre-copy approach to implement wide-area live virtual machine migration.

ACKNOWLEDGMENT

This paper is supported by the National High Technology Research and Development Program of China 2006AA01Z121.

REFERENCES

- [1] R. Goldberg, "Survey of virtual machine research," *IEEE Computer*, 1974, pp. 34–45.
- [2] C. Clark, K. Fraser, S. Hand, J. G. Hansen, E. Jul, C. Limpach, I. Pratt, and A. Warfield, "Live migration of virtual machines," in *Proceedings of the Second Symposium on Networked Systems Design and Implementation (NSDI'05)*, 2005, pp. 273–286.
- [3] M. Nelson, B. Lim, and G. Hutchins, "Fast transparent migration for virtual machines," in *Proceedings of the USENIX Annual Technical Conference (USENIX'05)*, 2005, pp. 391–394.
- [4] P. Barham, B. Dragovic, K. Fraser, S. Hand, T. Harris, A. Ho, R. Neugebauer, I. Pratt, and A. Warfield, "Xen and the art of virtualization," in *Proceedings of the 19th ACM symposium on Operating Systems Principles (SOSP'03)*, 2003, pp. 164–177.
- [5] A. Kivity, Y. Kamay, D. Laor, U. Lublin, and A. Liguori, "kvm: the linux virtual machine monitor," in *Proceeding of the 2007 Ottawa Linux Symposium*, 2007, pp. 225–230.
- [6] Container-based virtualization for linux, <http://www.openvz.com>.
- [7] R. Bradford, E. Kotsovinos, A. Feldmann, and H. Schioberg, "Live wide-area migration of virtual machines including local persistent state," in *Proceedings of the 3rd International Conference on Virtual Execution Environment (VEE'07)*, 2007, pp. 169–179.
- [8] Y. Luo, B. Zhang, X. Wang, Z. Wang, Y. Sun, and H. Chen, "Live and incremental whole-system migration of virtual machines using block-bitmap," in *Proceedings of the IEEE International Conference on Cluster Computing (Cluster'08)*, 2008, pp. 99–106.
- [9] M. R. Hines and K. Gopalan, "Post-copy based live virtual machine migration using adaptive pre-paging and dynamic self-ballooning," in *Proceedings of the ACM/Usenix International Conference on Virtual Execution Environments (VEE'09)*, 2009, pp. 51–60.
- [10] H. Liu, H. Jin, X. Liao, L. Hu, and C. Yu, "Live migration of virtual machine based on full system trace and replay," in *Proceedings of the 18th International Symposium on High Performance Distributed Computing (HPDC'09)*, 2009, pp. 101–110.
- [11] H. Jin, L. Deng, S. Wu, X. Shi, and X. Pan, "Live virtual machine migration with adaptive memory compression," in *Proceedings of the 2009 IEEE International Conference on Cluster Computing (Cluster 2009)*, 2009.
- [12] <http://mummer.sourceforge.net/>.
- [13] P. Riteau, C. Morin, T. Priol, "Shrinker: Efficient Wide-Area Live Virtual Machine Migration using Distributed Content-Based Addressing," <http://hal.inria.fr/docs/00/45/47/27/PDF/RR-7198.pdf>, 2010.